동국대학교
dongguk university

응용수치해석 및 실습 제15강

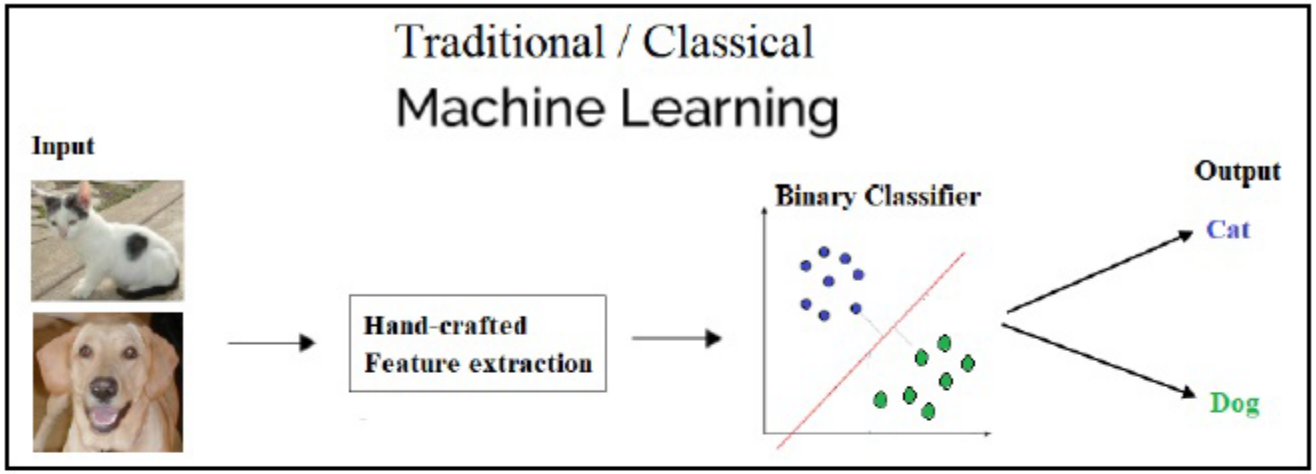
권기운

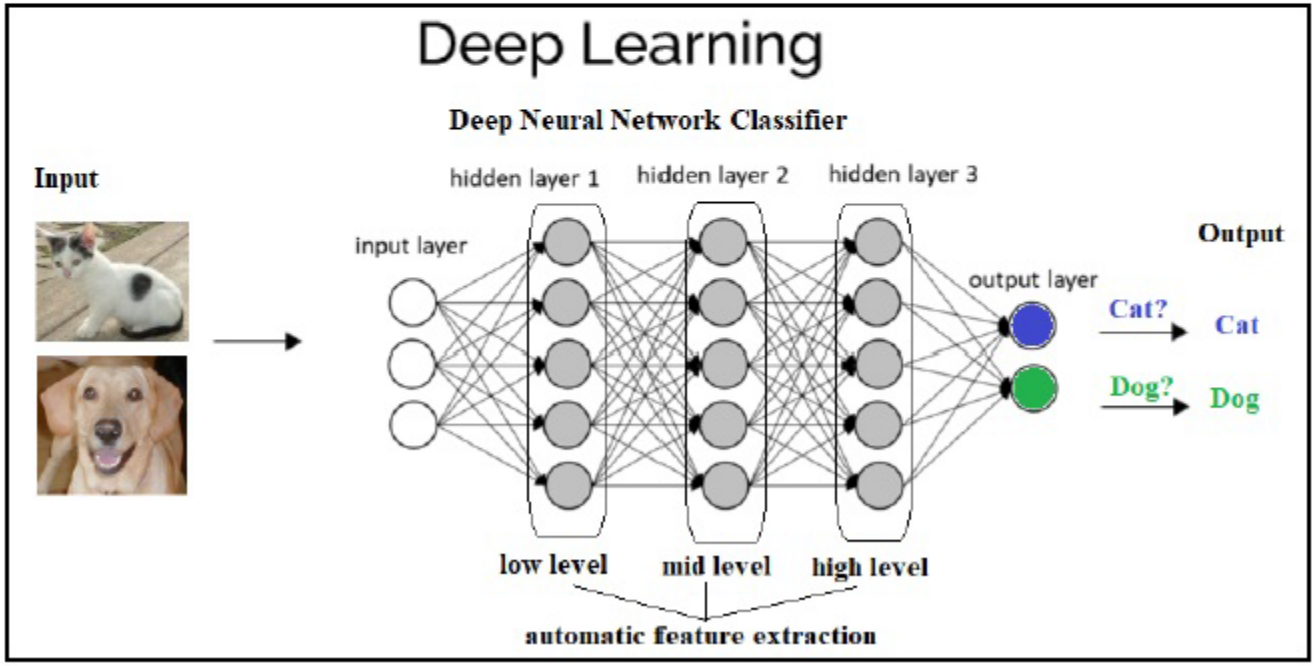
Chapter 10. 딥러닝 – 영상 분류

- 영상 처리에서 딥러닝
- CNNs
- Tensorflow 또는 Keras로 MNIST 영상 분류
- 인기 있는 심층 CNN

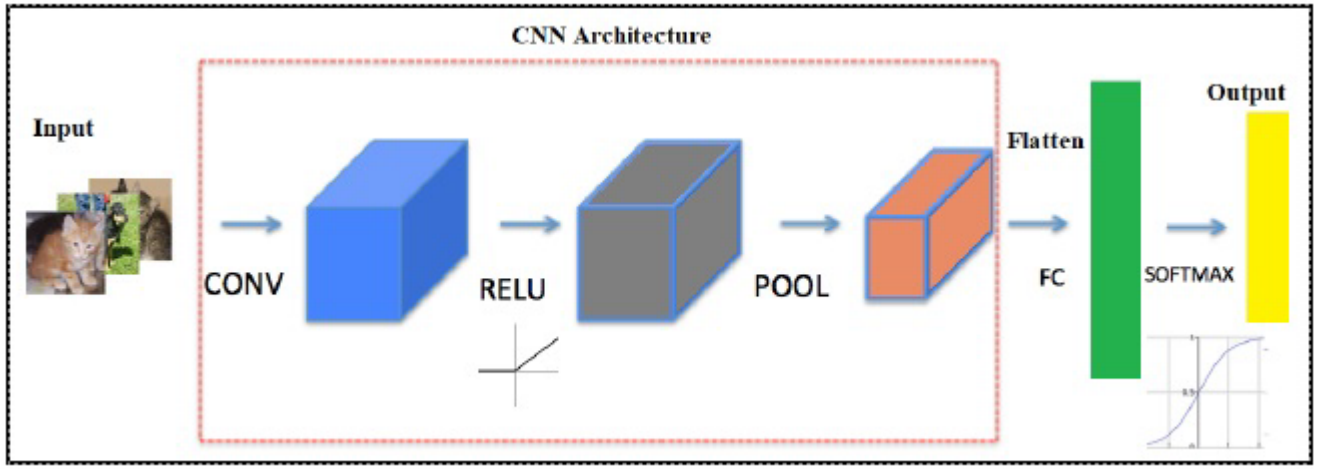
기계학습과 심층학습

- ML : 특징 추출과 분류자
- DL : 은닉층과 의사결정





CNN : Convolutional Neural Network



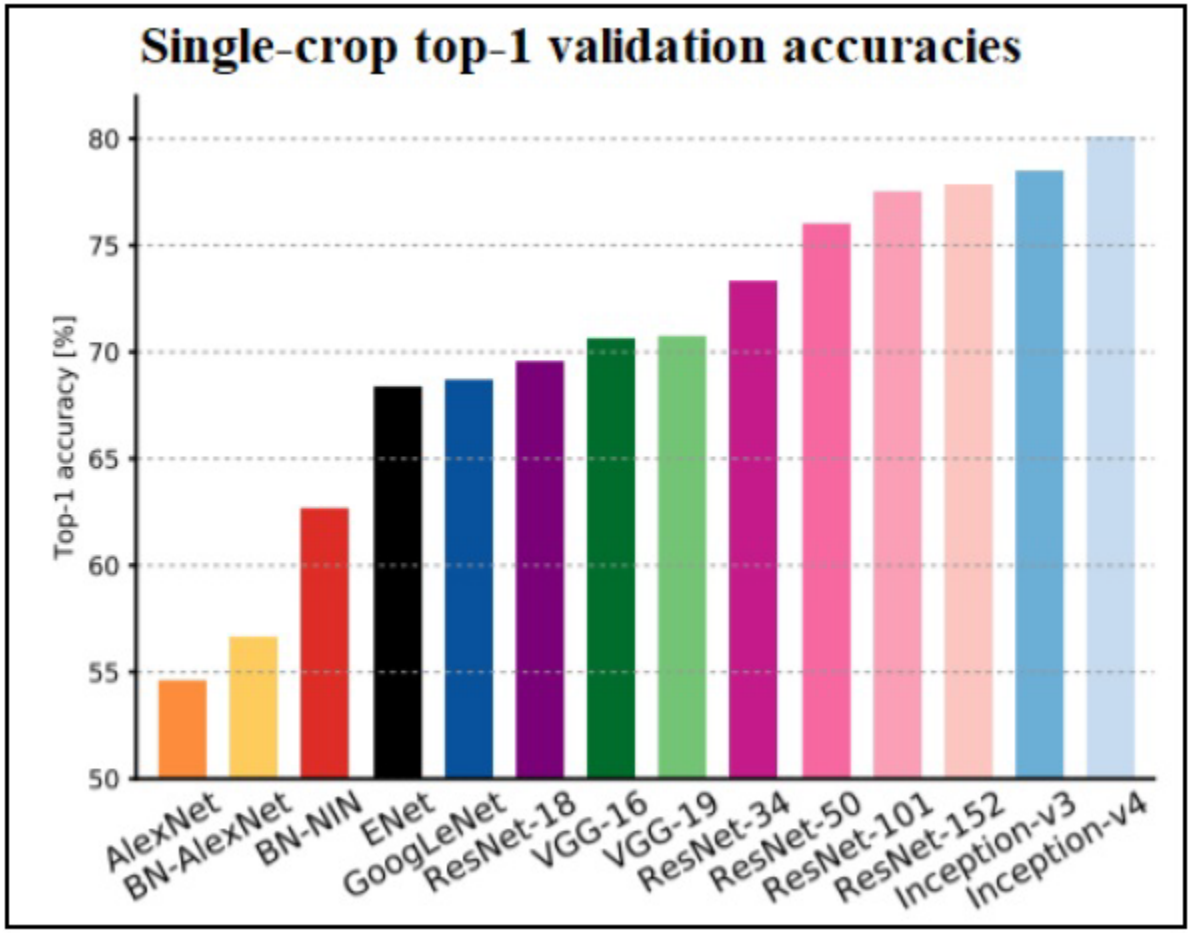
- Convolution 계층, 비선형 ReLU 활성화 계층, Pooling 계층, 전체 연결 Fully Connected 계층, FC softmax 계층
- 영상분류를 위해 설계된 CNN
- 다중 컨벌루션, ReLU, 풀링 연속 계층 : 신경망을 더 깊게 만들어 복잡한 영상처리 가능

Tensorflow 또는 Keras(Tensorflow.keras)로 영상 분류

- MNIST 필기 숫자 분류 문제를 해결
- 두 가지 유명한 딥러닝 라이브러리
- Tensorflow TF : 가장 유명
  - 매우 크고 멋진 커뮤니티
  - 사용하기가 쉽지 않다
  - 저수준 라이브러리 제공이 Keras 보다 더 유연
- Keras : TF 기반 고급 API
  - 저수준 아키텍쳐 제어 미흡
  - TF와 비교하여 사용하기 쉬움
  - 사용자 친화적

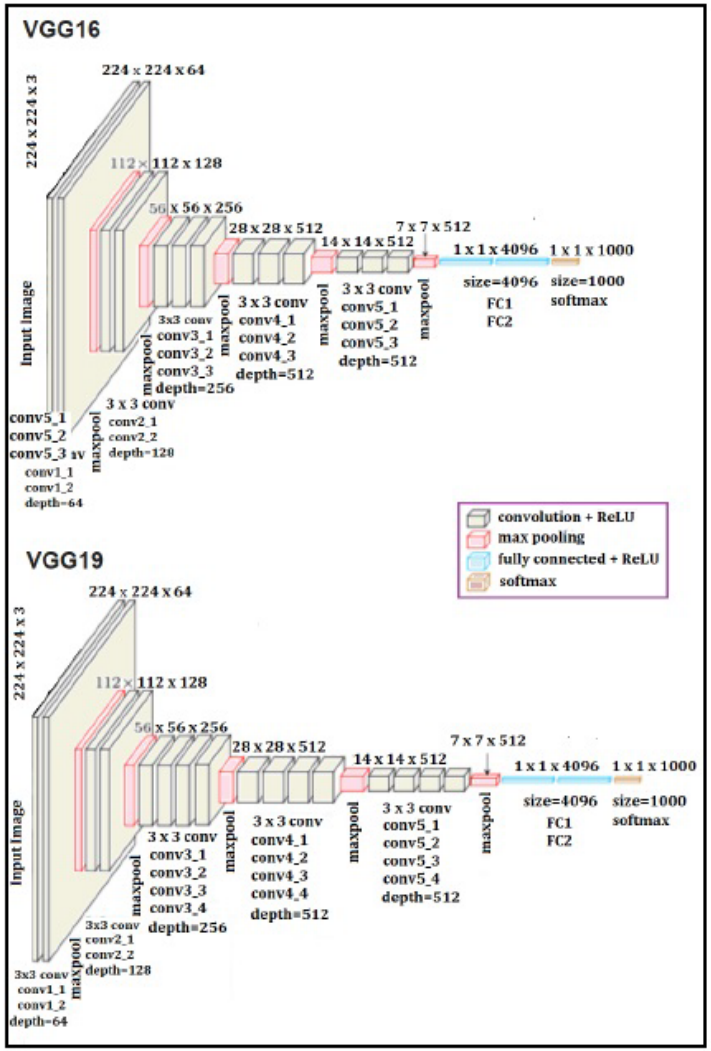
인기있는 심층 CNN

- VGG-16/19, ResNet, InceptionNet
- ImageNet Challenge에 제출 된 연도별 가장 성능 좋은 출품작의 최고 판단(single crop) 정확도 : AlexNet(Krizhevsky 외, 2012), InceptionNet(Szegedy 외, 2016)



VGG-16/19

- VGG-16/19
  - 아주 단순한 네트워크
  - 컨벌루션 : 보폭 1인 3*3필터만 사용
  - 풀링 : 보폭 2인 2*2 최댓값 풀링, 동일 패딩
  - 1억 3천 8백만개의 매개 변수
- VGG-16을 이용한 개/고양이 분류
  - Kaggle의 개/고양이 대회 영상 분류
    - Dogs vs. Cats competition
  - 학습과 영상 데이터 셋 다운로드
    - https://www.kaggle.com/c/dogs-vs-cats
  - 가중치가 없는 데서부터 학습
    - Weights = None
  - 훈련 단계(training page)
    - 학습 데이터 2만개
    - 학습 1만5천개, 검증 5천개
    - GPU : 시간 절약
    - Epochs=20
    - 훈련 정확도 73.65%, 검증 정확도 71.82%
    - 첫 블록/두번째 블록 두번째 컨볼루션 첫 64개 필터 개 영상 시각화
  - 예측 단계(Testing page)
    - 첫 100개 테스트 영상 예측한 클래스와 확률
    - 잘못 예측 영상 조금 있지만, 대부분 맞음



Example 4. VGG16 : 모델 정보

Model: "vgg16"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 50, 50, 3)	0
block1_conv1 (Conv2D)	(None, 50, 50, 64)	1,792
block1_conv2 (Conv2D)	(None, 50, 50, 64)	36,928
block1_pool (MaxPooling2D)	(None, 25, 25, 64)	0
block2_conv1 (Conv2D)	(None, 25, 25, 128)	73,856
block2_conv2 (Conv2D)	(None, 25, 25, 128)	147,584
block2_pool (MaxPooling2D)	(None, 12, 12, 128)	0
block3_conv1 (Conv2D)	(None, 12, 12, 256)	295,168
block3_conv2 (Conv2D)	(None, 12, 12, 256)	590,080
block3_conv3 (Conv2D)	(None, 12, 12, 256)	590,080
block3_pool (MaxPooling2D)	(None, 6, 6, 256)	0
block4_conv1 (Conv2D)	(None, 6, 6, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 6, 6, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 6, 6, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 3, 3, 512)	0
block5_conv1 (Conv2D)	(None, 3, 3, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 3, 3, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 3, 3, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 1, 1, 512)	0
flatten (Flatten)	(None, 512)	0
fc1 (Dense)	(None, 4096)	2,101,248
fc2 (Dense)	(None, 4096)	16,781,312
predictions (Dense)	(None, 2)	8,194

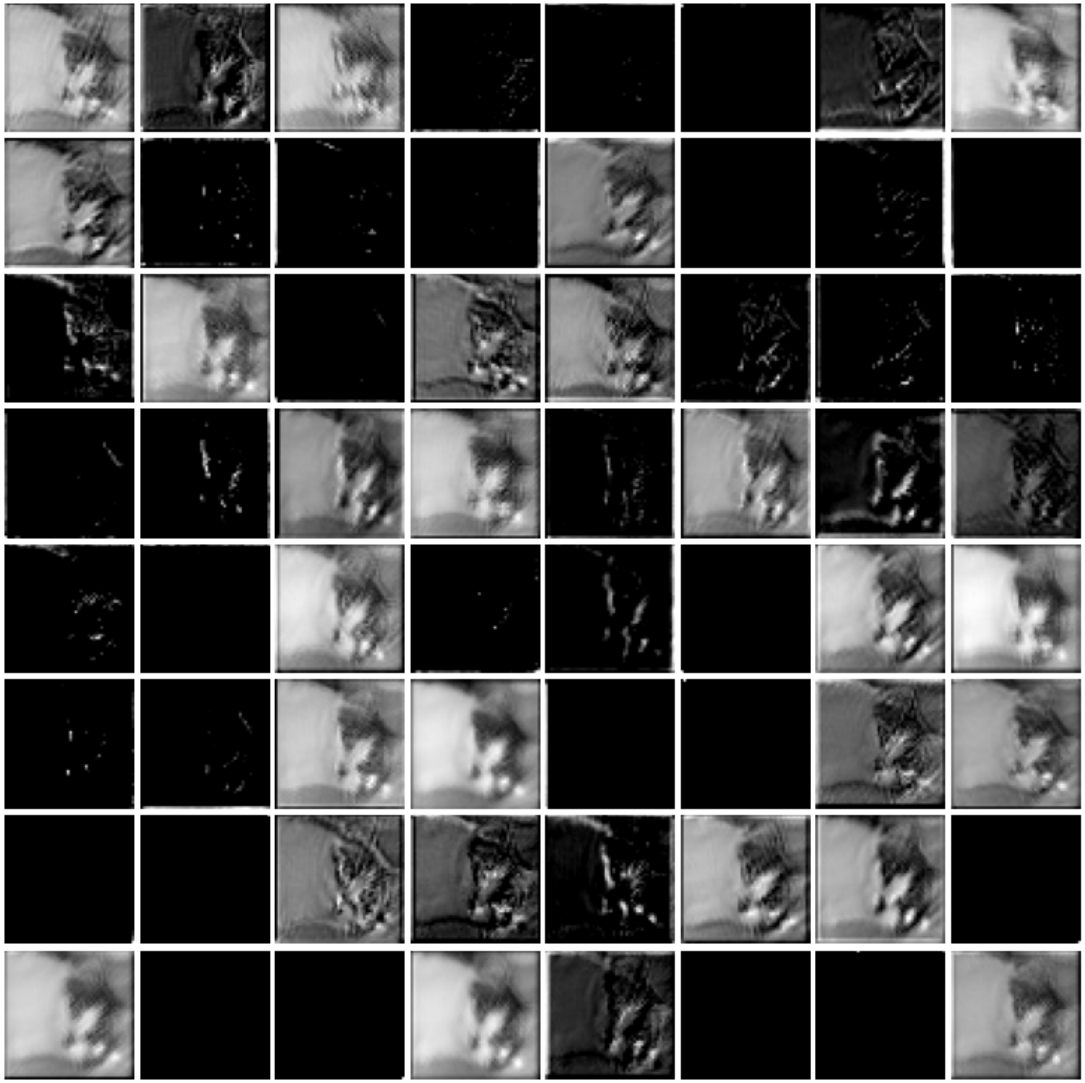
Total params: 33,605,442 (128.19 MB)
Trainable params: 33,605,442 (128.19 MB)
Non-trainable params: 0 (0.00 B)

Example 4 : Training : Accuracy: 0.72 Error: 28%

```
Epoch 1/20
79/79 - 300s - 4s/step - accuracy: 0.5495 - loss: 0.6874 - val_accuracy: 0.5958 -
val_loss: 0.6797
Epoch 2/20
79/79 - 356s - 5s/step - accuracy: 0.6072 - loss: 0.6731 - val_accuracy: 0.6136 -
val_loss: 0.6667
Epoch 3/20
79/79 - 376s - 5s/step - accuracy: 0.6250 - loss: 0.6570 - val_accuracy: 0.6234 -
val_loss: 0.6525
Epoch 4/20
79/79 - 373s - 5s/step - accuracy: 0.6431 - loss: 0.6390 - val_accuracy: 0.6350 -
val_loss: 0.6343
Epoch 5/20
79/79 - 387s - 5s/step - accuracy: 0.6552 - loss: 0.6246 - val_accuracy: 0.6446 -
val_loss: 0.6264
Epoch 6/20
79/79 - 385s - 5s/step - accuracy: 0.6653 - loss: 0.6131 - val_accuracy: 0.6636 -
val_loss: 0.6126
Epoch 7/20
79/79 - 405s - 5s/step - accuracy: 0.6791 - loss: 0.6019 - val_accuracy: 0.6664 -
val_loss: 0.6048
Epoch 8/20
79/79 - 390s - 5s/step - accuracy: 0.6801 - loss: 0.5967 - val_accuracy: 0.6706 -
val_loss: 0.6006
Epoch 9/20
79/79 - 425s - 5s/step - accuracy: 0.6860 - loss: 0.5896 - val_accuracy: 0.6806 -
val_loss: 0.5928
Epoch 10/20
79/79 - 557s - 7s/step - accuracy: 0.6928 - loss: 0.5825 - val_accuracy: 0.6834 -
val_loss: 0.5866
Epoch 11/20
79/79 - 556s - 7s/step - accuracy: 0.7029 - loss: 0.5751 - val_accuracy: 0.6872 -
```

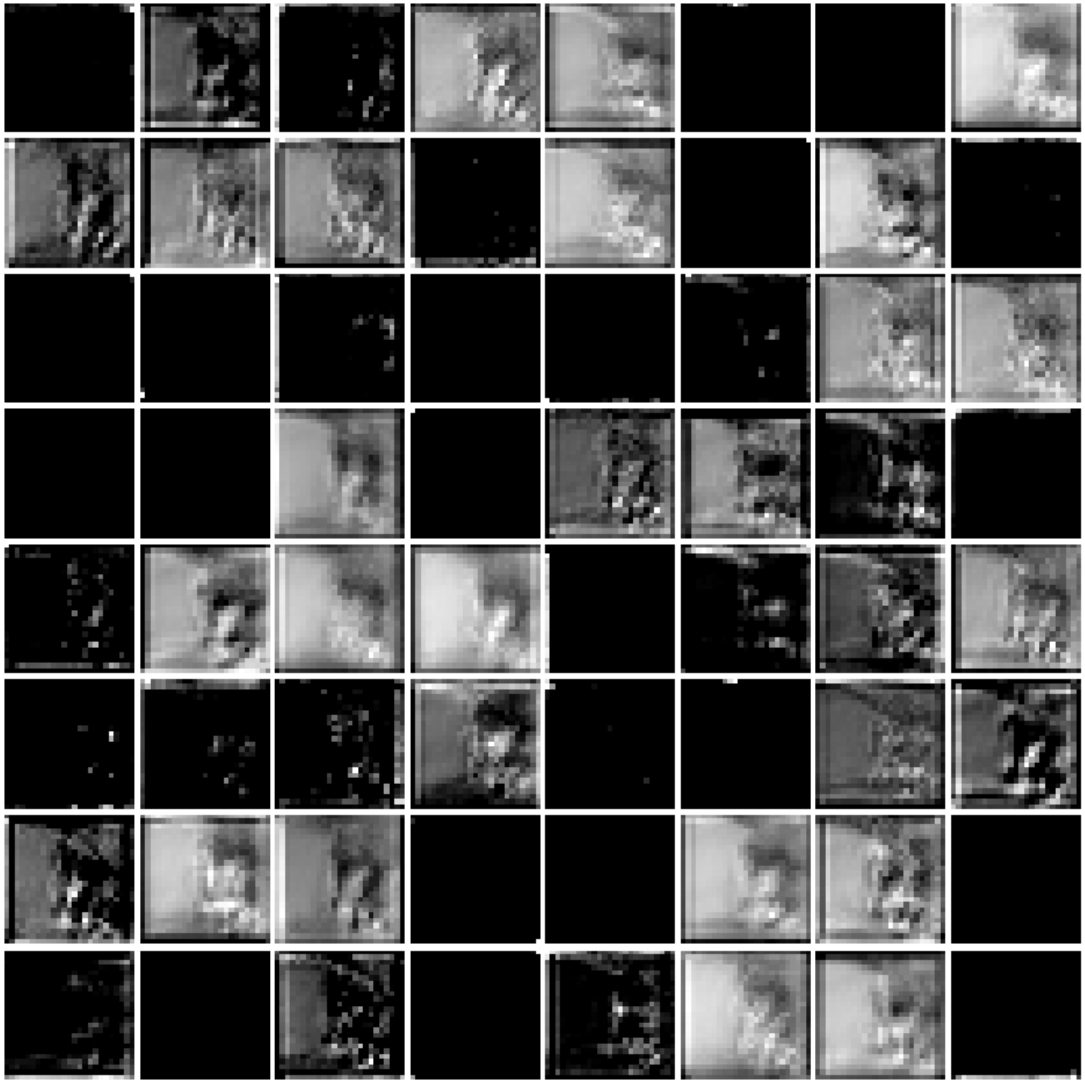
```
val_loss: 0.5866
Epoch 11/20
79/79 - 556s - 7s/step - accuracy: 0.7029 - loss: 0.5751 - val_accuracy: 0.6872 -
val_loss: 0.5831
Epoch 12/20
79/79 - 555s - 7s/step - accuracy: 0.7070 - loss: 0.5686 - val_accuracy: 0.6936 -
val_loss: 0.5767
Epoch 13/20
79/79 - 556s - 7s/step - accuracy: 0.7112 - loss: 0.5629 - val_accuracy: 0.6916 -
val_loss: 0.5772
Epoch 14/20
79/79 - 547s - 7s/step - accuracy: 0.7167 - loss: 0.5570 - val_accuracy: 0.7012 -
val_loss: 0.5685
Epoch 15/20
79/79 - 547s - 7s/step - accuracy: 0.7129 - loss: 0.5593 - val_accuracy: 0.6990 -
val_loss: 0.5701
Epoch 16/20
79/79 - 546s - 7s/step - accuracy: 0.7274 - loss: 0.5457 - val_accuracy: 0.7022 -
val_loss: 0.5664
Epoch 17/20
79/79 - 553s - 7s/step - accuracy: 0.7298 - loss: 0.5409 - val_accuracy: 0.7028 -
val_loss: 0.5652
Epoch 18/20
79/79 - 361s - 5s/step - accuracy: 0.7297 - loss: 0.5375 - val_accuracy: 0.7134 -
val_loss: 0.5551
Epoch 19/20
79/79 - 377s - 5s/step - accuracy: 0.7388 - loss: 0.5307 - val_accuracy: 0.6988 -
val_loss: 0.5663
Epoch 20/20
79/79 - 379s - 5s/step - accuracy: 0.7365 - loss: 0.5265 - val_accuracy: 0.7182 -
val loss: 0.5449
```

Example 4 : 중간계층 시각화 : block1_conv2 (데이터 1/10)

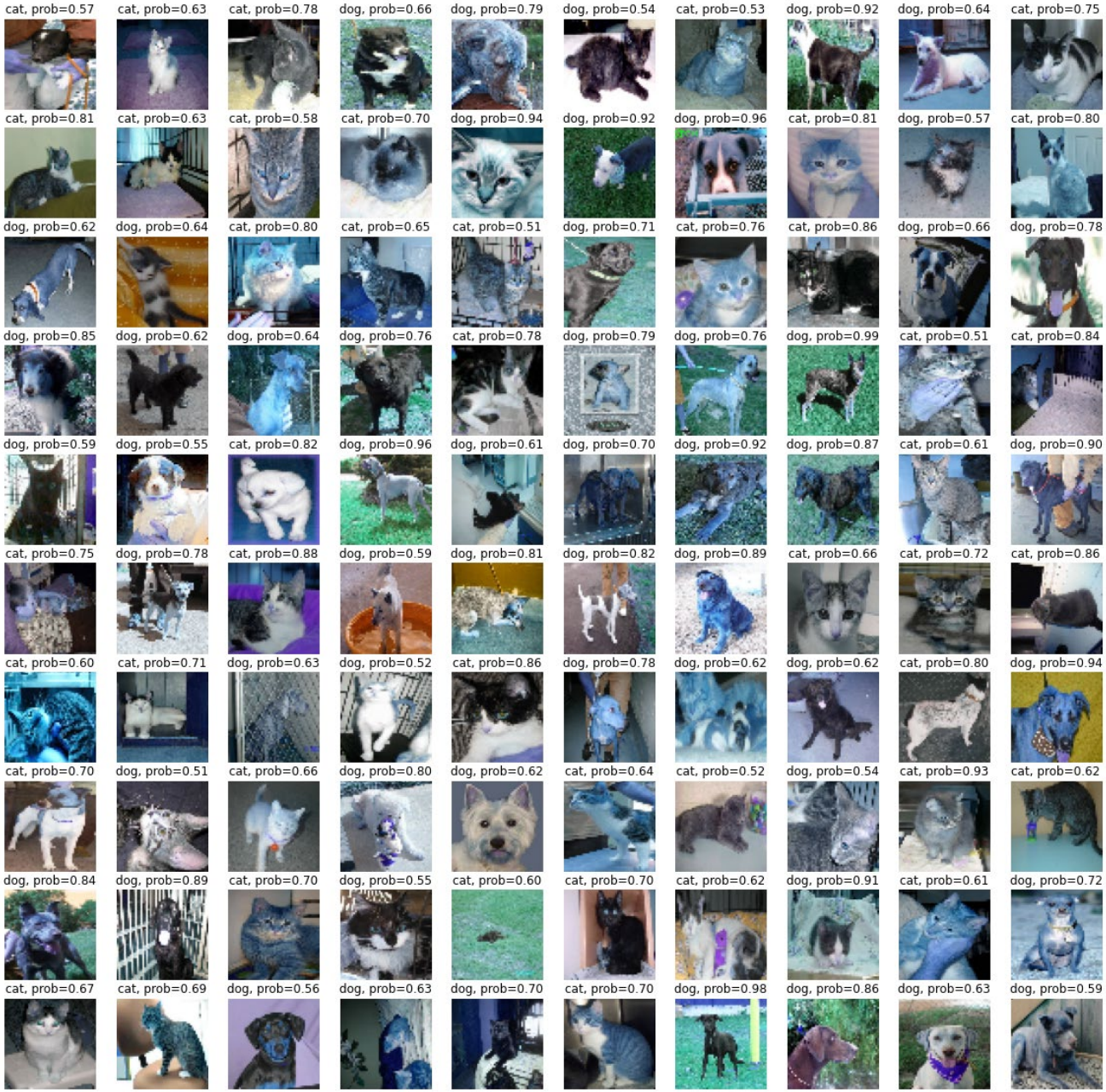


동국대학교
dongguk university

Example 4 : 중간계층 시각화 : block2_conv2 (데이터 1/10)

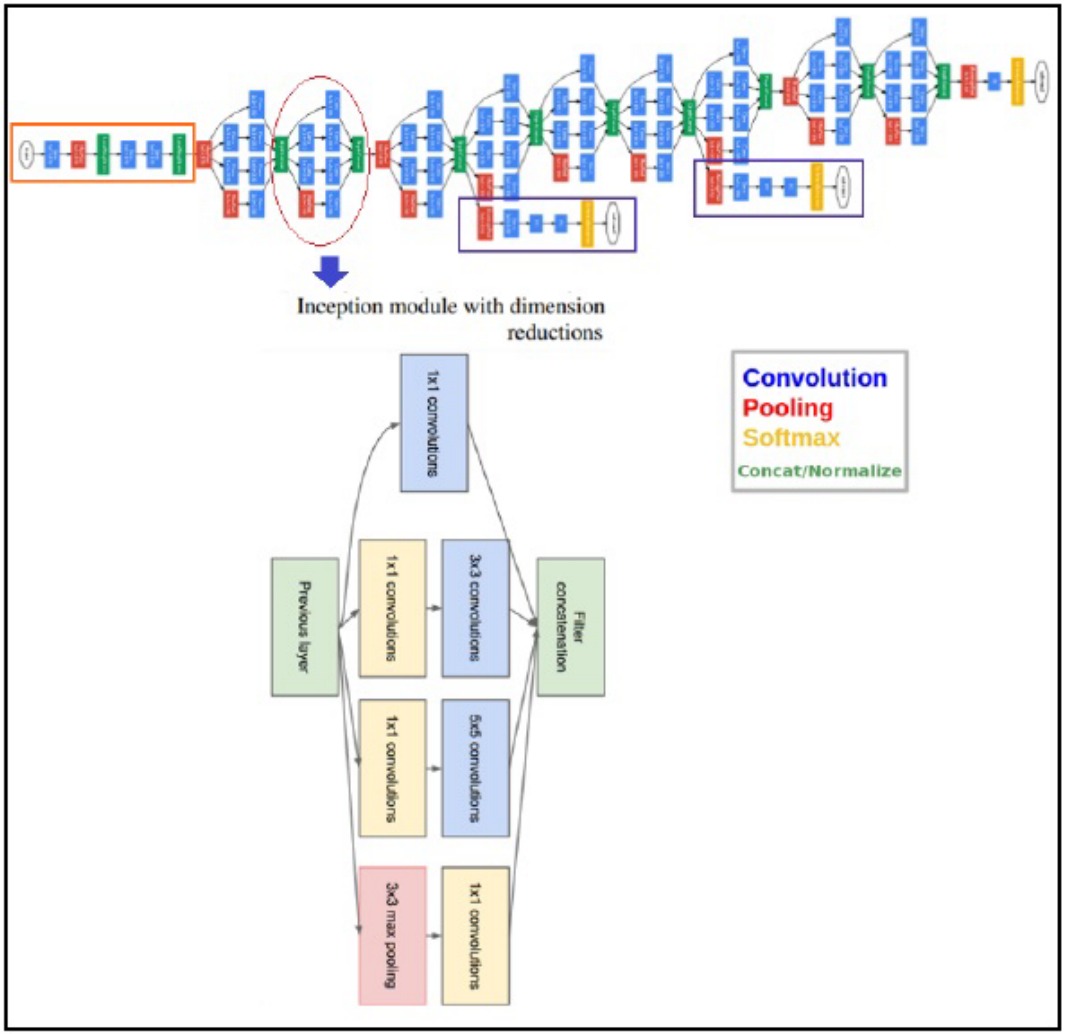


Example 4 : Test



동국대학교
dongguk university

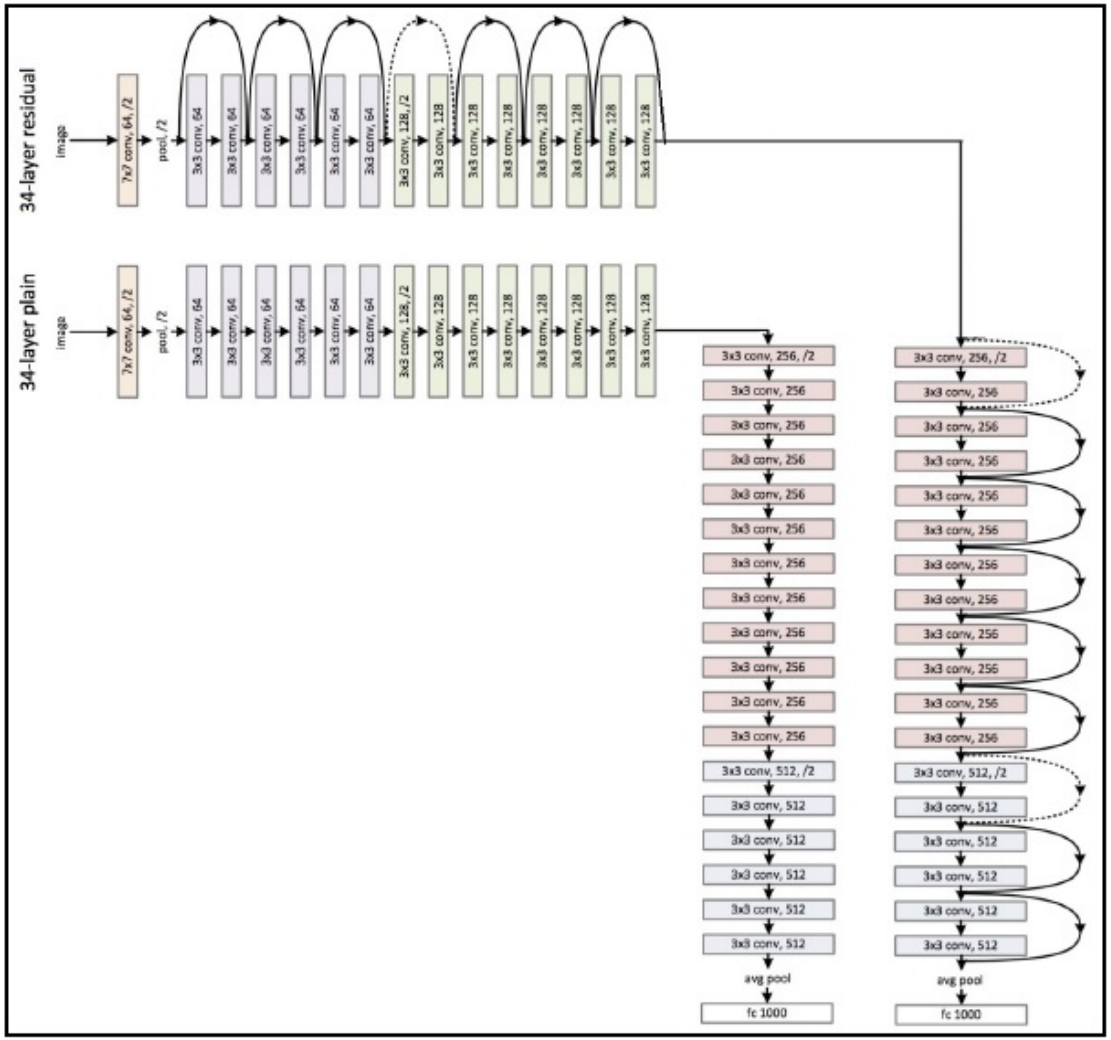
인셉션넷(InceptionNet)



인셉션 넷

- CNN 분류기 개발에서 중요한 이정표
- 이전 CNN은 더 나은 성능을 위해 컨볼루션 계층을 극한의 깊이까지 쌓아 옴
- 속도와 정확성을 위해 복잡한 기술과 기법 사용
- 끊임없이 진화함, 여러 가지 버전
- 인기있는 버전 : Inception-v1, v2, v3, v4, Inception-ResNet
- 컨볼루션 작업에 적합한 커널 크기를 선택하는 것이 까다로움
- 커널이 커지면 전역적 정보, 작아지면 지역적 정보 처리
- 심층 신경망은 과적합 overfitting과 소멸 vanishing gradient 문제
  - 대규모 컨볼루션을 단순히 쌓아 올리면 많은 비용
- 동일한 레벨에서 작동하는 여러 크기의 필터를 추가하여 과적합과 소멸 문제 해결
- 네트워크가 깊어지는 대신 폭이 넓어짐
- 차원 감소가 있는 인셉션 모듈
  - 3가지 다른 크기 1*1, 3*3, 5*5 필터
  - 최댓값 풀링
- 비용을 줄이기 위해 3*3, 5*5 컨볼루션 전에 1*1 컨볼루션 추가 : 입력 채널 수 제한
- GoogleNet(Inception-v1)
  - 선형으로 쌓인 9개의 인셉션 모듈
  - 22층(풀링 포함 시 27층)
  - 마지막 인셉션 모델 끝에 전역 평균값 풀링

ResNet


34-layer plain
34-layer residual
image
7x7 conv, 64, /2
pool, /2
3x3 conv, 64
3x3 conv, 128, /2
3x3 conv, 128
3x3 conv, 256, /2
3x3 conv, 256
3x3 conv, 512, /2
3x3 conv, 512
avg pool
fc 1000

ResNet, Residual Network

- 단순히 계층을 쌓기만 한다고 해서 네트워크가 깊어지지는 않는다.

- 기울기 소멸 문제 Vanishing gradient problem
  - 기울기가 이전 계층으로 역전파될 때, 반복되는 역전파로 인해 기울기가 무한히 작아질 수 있음
  - 계층이 깊어짐에 따라 성능에 영향

- 지름길 연결 Shortcut connection : 네트워크에 지름길 도입
  - 하나 이상의 계층을 건너뛰는 작업 수행 : 쌓인 계층의 기울기가 작아지는 것 방지
  - 쌓인 식별(identity) 계층은 현재 네트워크에서 단순히 매핑을 쌓는 것 외에는 다른 수행 없음

- 깊은 모델에서 기울기 소멸을 방지하여 얕은 모델보다 성능을 높임

- ResNet 34계층

- Keras는 ResNet50 모델 제공
  - 가중치 없이 처음부터 학습
  - 사전학습 네트워크에서 로드된 가중치로부터 학습도 가능

결론

- 딥러닝 모델을 사용한 영상처리의 최근 발전 소개하였다.
- 딥러닝 기본 개념, 전통적인 ML과 다른 점, 딥러닝이 필요한 이유에 대해 설명했다.
- 복잡한 영상처리 및 컴퓨터 비전 작업을 해결하기 위해 특별히 고안된 심층신경망으로서 CNN 소개하였다.
- 컨볼루션, 풀링, FC 계층이 있는 CNN 아키텍쳐를 논의하였다.
- 파이썬에 널리 사용되는 두가지 라이브러리 Tensorflow와 Keras를 알아보았다.
- 필기숫자분류에서 MNIST 데이터 셋의 테스트 정확도를 증가시키기 위해
  - FC계층만을 사용하는 방법
  - CNN을 사용하는 방법

  을 보여주었다.
- VGG-16/19, GoogleNet, ResNet과 같은 인기있는 네트워크에 대해 살펴보았다.
- Keras의 VGG-16모델은 Kaggle의 개 대 고양이 대회 영상으로 학습하고, 검증 데이터 셋에서 적절한 정확도를 실행되는가 확인하였다.

동국대학교
dongguk university